Universidad Nacional de Colombia

Sede Medellín

Ingeniería Ontológica – 3010090

Trabajo Práctico 2 - Valor: 25%

Knowledge Graph RAG: RAG Avanzado + Grafo de Conocimiento

Profesor: Jaime Alberto Guzmán Luna

Fecha de Entrega: viernes 10 de abril, Hora de cierre: 11:59 pm.

INTRODUCCIÓN

Esta práctica tiene como fin la aplicación de técnicas avanzadas de Recuperación Aumentada por Generación (RAG) y la orquestación de flujos mediante agentes autónomos. El estudiante debe seleccionar un dominio de aplicación y construir un sistema RAG agéntico orientado al análisis inteligente de documentos, incorporando estrategias avanzadas de recuperación, grafos de conocimiento y trazabilidad profunda.

OBJETIVO GENERAL

Diseñar e implementar un sistema RAG agéntico capaz de responder preguntas complejas en lenguaje natural, inspirado en los modelos ReAct y Agente Reflexivo (Reflecting). El sistema debe ser construido utilizando LangGraph para la orquestación del flujo, integrando técnicas de recuperación avanzada (MMR, HyDE y Descomposición de Consultas), una ontología OWL y sus instancias para crear un sistema Knowledge Graph RAG que permite la trazabilidad completa del razonamiento mediante el debug de LangSmith y la trazabilidad de donde la información fue recuperada.

OBJETIVOS DE APRENDIZAJE

1. Orquestar flujos de trabajo de agentes autónomos y reflexivos (Reflecting) implementando grafos de ejecución con LangGraph.
2. Implementar estrategias de indexación y recuperación avanzada, incluyendo Chunking Semántico y Búsqueda Híbrida/MMR.
3. Aplicar técnicas de transformación de consultas como HyDE y Query Decomposition para optimizar la recuperación.
4. Integrar una ontología de dominio para extender el sistema hacia un Knowledge Graph RAG.
5. Registrar evidencia de cómo se produjo una respuesta y depurar el flujo de razonamiento (debug) utilizando LangSmith.
6. Almacenar la trazabilidad de la información, indicando de que documentos o lugar se extrajo la información para que el RAG diera su respuesta

DESCRIPCIÓN DETALLADA DEL SISTEMA A DESARROLLAR

PARTE A – AGENTIC RAG AVANZADO

1. **Construcción del Corpus**
   - Seleccionar un dominio con mínimo 50 documentos (PDF/TXT/HTML), tales como reglamentos, manuales técnicos, normas, artículos, etc. Esto difiere de la cantidad de chunks de datos generados.
   - Diseñar e implementar una ontología básica representativa del dominio seleccionado, la cual servirá como base para el Knowledge Graph. Ver los detalles en la Parte B.
2. **Consumo, Indexación y Chunking Semántico**
   - Cargar los documentos (PDF/TXT/HTML) que conforman el dominio seleccionado.
   - Implementar **Chunking Semántico** en lugar de segmentación por tamaño fijo, agrupando el texto por significado y coherencia temática. Usar *SemanticChunker*.
   - Indexar información en una base vectorial (FAISS, Chroma u otra).
3. **Procesamiento y Transformación de Consultas**
   - El sistema debe analizar la consulta del usuario en lenguaje natural y aplicar transformaciones dinámicas:

    - **HyDE (Hypothetical Document Embeddings)**: Identificar preguntas cortas o ambiguas y generar un documento hipotético para mejorar la búsqueda semántica.
    - **Query Decomposition**: Detectar cuando una consulta contiene múltiples preguntas o condicionales, descomponiéndola en sub-consultas más simples que se resolverán de manera secuencial o en paralelo.

4. **Arquitectura del Agente (ReAct y Reflecting)**
    - Implementar un agente basado en el framework ReAct (Reasoning and Acting), capaz de decidir qué herramientas utilizar (búsqueda vectorial, consulta al grafo de conocimiento, descomposición de la pregunta, aplicar hyde, etc.) en función de la pregunta.
    - Incorporar un patrón de **Agente Reflexivo (Reflecting)**: el sistema debe generar una respuesta preliminar y luego autoevaluarla/criticarla. En caso de fallo o información incompleta, solicita nueva respuesta al agente RAG (loop controlado). Reintentar máximo 3 veces, luego de eso consultar la información en internet u otro recurso por medio de una tool.
5. **Búsqueda y Recuperación Avanzada**
    - **Búsqueda MMR (Maximal Marginal Relevance) o Híbrida:** Ejecutar la búsqueda semántica utilizando uno de los dos métodos a implementar: (1) MMR para garantizar relevancia y diversidad en los fragmentos recuperados, reduciendo la redundancia. (2) En caso de una búsqueda hibrida, justificar la elección de los pesos para la búsqueda Sparse vs Dense
    - **Grafo de Conocimiento (Knowledge Graph):** Proveer al agente de una herramienta que le permita consultar la ontología construida para extraer relaciones estructuradas y enriquecer el contexto, complementando la búsqueda vectorial tradicional (ver PARTE C de este enunciado).
6. **Trazabilidad con LangSmith**
    - El sistema debe estar conectado a **LangSmith**.
    - Se debe evidenciar el registro de ejecución de cada nodo de LangGraph, mostrando el flujo de pensamiento del RAG agentico, las transformaciones de las consultas, los documentos recuperados y el resultado de la verificación/crítica.
7. **Evaluación del Sistema**
    - Implementar métricas (las que sean posibles acorde a los anteriores requerimientos): Recall@k; Precision@k; MRR / nDCG; Relevance; Faithfulness.
    - Incluir evaluación con LLM como juez.

PARTE B – FLUJO GENERAL DEL SISTEMA RAG

1. Usuario ingresa la consulta.
2. Transformación de consulta: El sistema evalúa si aplica HyDE (pregunta corta) o Query Decomposition (pregunta múltiple).
3. Agente ReAct: Analiza la(s) consulta(s) y decide el plan de acción y las herramientas a usar.
4. Recuperación: Se extrae información usando MMR/Búsqueda Híbrida en la base vectorial y/o consultas estructuradas en la Ontología (GraphRAG).
5. Generación de Respuesta preliminar.
6. Reflexión/Crítica: Evaluación de la respuesta. Si la respuesta no es adecuada → regeneración de respuesta.
7. Calcular las métricas del sistema RAG
8. Se retorna la respuesta final al usuario
9. En caso de no encontrar una buena respuesta luego de 3 intentos, realizar la búsqueda con una tool en internet u otro recurso, responder y retroalimentar la base de conocimiento.

PARTE C – GRAFO DE CONOCIMIENTO (KNOWLEDGE GRAPH)

1. **Ontología OWL**
   - Crear una ontología en formato Turtle que incluya:
     - Clases: Implementar al menos 10 clases (owl:Class). Hacer uso de al menos dos casos de clases disjuntas (DisjointClasses)
     - Jerarquías: Crear 3 relaciones jerárquicas entre clases (rdfs:subClassOf) y 2 relaciones jerárquica entre propiedades (rdfs:subClassOf).
     - Propiedades: Crear al menos 10 propiedades originales con dominios (rdfs:domain) y rangos (rdfs:range) correctamente definidos. Incluir como rangos valores tanto de tipo clase (URIs) y otros Tipo Literales con sus respectivos tipados usando el XML-Schema.
     - Individuos: Instanciar al menos 4 individuos por clase
     - Relaciones de propiedad: Implementar al menos 2 casos de owl:inverseOf
     - Restricciones de propiedad: crear Al menos un caso de los siguientes elementos:
       - Restricción Universal: owl:allValuesFrom
       - Restricción existencial: owl:someValuesFrom

        - Restricciones de cardinalidad: seleccionar una de las siguientes (owl:cardinality, owl:minCardinality, owl:maxCardinality)
    - Constructores lógicos: seleccionar uno de las siguientes: owl:intersectionOf, owl:unionOf, owl:complementOf
2. **Grafo de Conocimiento**
    - Subir la Ontologia OWL creada en la base de conocimiento GraphDB.
    - Consultas SPARQL: Implementar al menos un caso de cada una de las siguientes consultas:
        - SELECT
        - - FILTER
        - - ORDER BY
        - - LIMIT
        - – UPDATE (al menos 2 de los operandos)

        Implementar los casos con RDFLib conectándose a GraphDB

    - Inferencias: Documentar al menos 5 casos de inferencia sobre la ontología habilitándole la opción de inferencia a GraphDB. Para ello generar las consultas con RDFLib y SPARQL conectándose a GraphDB.
3. **Knowledge Graph RAG**
    - Implementar arquitectura que combine:
        - vector store
        - ontología OWL en el GraphDB
        - Consultas SPARQL
        - LLM
        - agente

PARTE D – TRAZABILIDAD CON LANGSMITH

1. El sistema debe estar conectado a LangSmith.
2. Se debe evidenciar el registro de ejecución de cada nodo de LangGraph, mostrando el flujo de pensamiento del RAG agentico, las transformaciones de las consultas, los documentos recuperados y el resultado de la verificación/crítica.
3. Procesar y analizar documentos reales.

ENTREGABLES

1. Código fuente de la implementación completa. Deberá incluir comentarios por función y explicación del flujo general.
2. Carpeta con documentos (PDF/TXT/HTML) utilizados: 50.
3. Informe técnico PDF detallado del sistema Knowledge Graph RAG que incluya:
    a. Diseño del sistema y registros de ejecución. Documentación de al menos 3 casos de uso que muestren las funcionalidades solicitadas del sistema.
    b. Las evaluaciones solicitadas (métricas y su justificación)
    c. Análisis de trazas en LangSmith
4. Ontología: archivo en formato turtle (.ttl)
5. Consultas SPARQL: documentar las consultas requeridas explícitamente
6. Inferencia con la ontología e instancias: documentar las inferencias explícitamente solicitadas.

SUSTENTACIÓN

1. Se deberá implementar un video tipo pitch (duración máx. 10 minutos) con la demostración del sistema en cada uno de los apartados relacionados en el documento de la práctica.
2. Todos los integrantes deben participar en el video.
3. El video deberá ser subido a YouTube y en el documento de técnico (PDF) a entregar incluir el enlace de este
4. Este video deberá estar disponible desde la fecha de entrega del trabajo indicada al inicio de este documento.

METODOLOGÍA DE EVALUACIÓN

1. Código: 70%
    a. Recuperación avanzada (numerales 1 al 3 y 5): 15%
    b. Agentes ReAct y Reflexivo (Reflecting) (numeral 4): 15%
    c. Knowledge Graph (Ontología): 15%
    d. Integración (Búsqueda semántica y Knowledge Graph): 10%
    e. Implementación del módulo para trazabilidad con LangSmith (numeral 6) :10%
    f. Evaluación del Sistema mediante métricas (numeral 7): 5%
2. Documentación (Informe técnico y análisis de trazas en LangSmith): 20%
3. Sustentación: 10%

MATERIAL POR ENTREGAR

Se deberá entregar en Google Classroom un archivo ZIP (con el siguiente nombre del archivo: practica2–equipo-YY.zip, donde YY el número del equipo) que contenga lo siguiente:

- **Informe técnico:** Descripción detallada del sistema RAG, junto con la justificación de las decisiones tomadas en cada parte del programa. Adicionalmente la dirección del pitch de la sustentación (video).
- **Código Fuente:** Todo el programa en Python, debidamente organizado y documentado.
- **Carpeta con documentos:** Documentos (PDF/TXT/HTML) utilizados: 50 y ontología .ttl.

ANEXO 1

INSTRUCCIONES GENERALES PARA LA PRESENTACIÓN DE LOS TRABAJOS

- Los trabajos serán presentados en grupos de 3 personas definidos al inicio del semestre (ver grupos en el material de la semana 2).
- No se admiten trabajos similares (incluyendo el planteamiento del dominio), en caso de presentarse programas con códigos similares serán anulados.
- La totalidad del trabajo solicitado al alumno, en esta evaluación práctica, deberá ser original y propio del autor que lo presenta.
- La sustentación del trabajo se realizará mediante un pitch de 10 minutos por parte de todos los estudiantes del grupo.

NOTA: La redacción de este documento de práctica puede presentar algunos errores o la ausencia de algunos requisitos específicos, los cuales deberán ser informados por parte de los estudiantes al profesor, para ser aclarados de manera colectiva durante la clase. Estas aclaraciones harán parte de los requisitos de la práctica.